Control of UML diagrams in designing automated systems software

A.N. Afanasyev, N.N. Voit, E. Yu. Voevodin
Ulyanovsk State Technical University
Ulyanovsk, Russia
{a.afanasev, n.voit}@ulstu.ru, voevodineu@ido.ulstu.ru

R.F. Gainullin
Ltd «Ecwid»
Ulyanovsk, Russia
r.gainullin@gmail.com

***Abstract*—The syntax-oriented methods for the syntax and semantic analysis and control of UML models in designing automated systems software are pro-posed. The methods are based on author automaton graphical grammars.**

***Index Terms*—Diagrammatic models, graphical automaton grammars, semantic errors.**

I. Introduction

The visual diagrammatic models of artifacts of business-processes are very important for automated systems software (ASS) design especially at the conceptual stages. That's why visual languages (UML, IDEF, eEPC, BPMN, etc.) have been developed and widely used in practice.

The usability of these models increases the efficiency of ASS design, the quality of design decisions, and the success of development as a whole by unifying the language of interaction between participants of the process of automated system (AS) design, documentation of solutions, and formal control of diagrammatic models [1].

However the modern theory of visual languages doesn't have the effective methods of analysis and control (especially with regard to control) of syntax and semantic errors of diagrammatic models.

Direct control methods requiring multiple passes implemented in the tool sets supporting the automated systems software design (e.g., RUP, ARIS) allow one to analyze the syntax correctness of diagrams and find out main errors. However they can't detect errors of so-called "separate contexts" related to the use of connectors such as "AND", "OR", "XOR", etc. The context-sensitive semantic errors connected with the text of any diagram are out of control. It is the reason of "expensive" errors in ASS design. Analysis and control of these errors are an important scientific and technical challenge.

II. Problem analysis

The purpose of the study is to expand the class of diagnosed errors.

The most common in industrial practice of designing the AS is the Unified Modeling Language (UML) used for the business-processes description at all stages of automated systems software design. It has been chosen as the object of the study.

To represent the diagrammatic models the modern theory of graphical and visual languages considers two basic syntax models called spatial and logical models. They are based on the attributes of graphic objects and types of their links.

The spatial model contains relative or absolute coordinates of graphic objects. The use of such model to control and analyze structural, i.e. topological (syntax), features of diagrams is very difficult.

Therefore to describe the syntax of diagrams the logical model is used. This model is processed by the graphical grammars.

John L. Pfaltz and Azriel Rosenfeld proposed web-grammars [2] of two-dimensional generating type.

Zhang developed the positional graphical grammar [2] relating to the context-free grammar, later it was developed by Costagliola [3]. Wittenberg and Weitzman [4] developed a relational graphical grammar. Zhang and Orgun [5, 6] described preserving graphical grammar in their works. The mentioned methods have the following shortcomings.

1. Positional grammars developing on the base of plex-structure don't use attaching points and can't be used for graphical languages the objects of which have dynamically changeable number of inputs\outputs.
2. Relational grammars generate a non-exhaustive list of errors because of the incompletely-engineered mechanism of neutralization.
3. There is no semantic control of textual attributes of diagrammatic models.
4. The common shortcomings of above mentioned grammars are: when designing the grammar for unstructured visual languages the rules increase (on a fixed quality of primitives, the significant quantity of rules increases to describe all the non-structural variants), the complexity of constructing the grammar. It takes much time to analyze (analyzers built on the basis of the considered grammars have polynomial or exponential time of diagram analysis of graphical languages).

III. Multi-level RVM-grammars

The methods of the UML diagrams analysis and control are based on the usability of RV-grammars.

When collective designing diagrammatical models have a complex hierarchical structure (complex diagrams), and they

increase the number of terms in many times. Classical graphical RV-grammar becomes more complicated, the process of its development becomes more sophisticated. The control of interconnected nodes and diagrams of complex models is not provided.

To exclude these shortcomings the multi-level grammar is proposed.

Consider multi-level system of RV-grammar given as a sequence of four elements:

$$\mathrm{RVM} =< n, \Sigma^n, \mathrm{RV}^n, r_0 >,$$

Where n is a grammar index; Σ^n is an alphabet of the n-th grammar; RV^n is a set of the n-th grammar rules; r_0 is an axiom of upper-level grammar.

As one of the states RV^igrammar contains RV^jgrammar. RV^jgrammar can also be compound.

$$G = (V, \Sigma, \tilde{\Sigma}, R, r_0),$$

RV^n– the L (R) language grammar is an ordered set of five nonempty sets, where $V = \{v_l, l = \overline{1, L}\}$ is an alphabet of operations over the internal memory; $\Sigma = \{a_t, t = \overline{1, T}\}$ is a terminal alphabet of the graphical language (the set of the graphical language primitives); $\tilde{\Sigma} = \{\tilde{a}_t, t = \overline{1, \tilde{T}}\}$ is a quasi-terminal alphabet extending the terminal alphabet. The alphabet includes:

- quasi-terms of graphic objects that won't continue analysis,
- quasi-terms of graphic objects with more than one input,
- quasi-terms of links – marks with specific semantic differences defined for them;
- quasi-term for completing the analysis;
- $R = \{r_i, i = \overline{1, I}\}$ is the scheme of the grammar G (a set of names of complexes of rules, where each complex r_i consists of a subset P_{ij} of rules $r_i = \{P_{ij}, j = \overline{1, J}\}$);
- $r_0 \in R$ is an axiom of RV-grammar (the name of the initial complex of rules), $r_k \in R$ is a final complex of rules.

The rules of $P_{ij} \in r_i$ are given as:

$$\tilde{a}_t \xrightarrow{W\gamma(\gamma_1,\dots,\gamma_n)} r_m,$$

where $W_\gamma(\gamma_1, \dots, \gamma_n)$ – n-ary relation defining the form of operation on the internal memory depending on $\gamma \in \{0,1,2,3\}$; $r_m \in R$ is the name of the receiver-rules.

The internal memory consists of stacks for processing the graphic objects that have more than one output, and elastic tapes for processing the graphic objects that have more than one input.

The RVM-system receives the symbols of terminal alphabet from input tape and transmits them to the appropriate level. The elements that transmit the grammar to another level are called "subterms". Then the RV^n-grammar description is given as:

$$G = (V, \Sigma, \tilde{\Sigma}, \bar{\Sigma}, R, r_0),$$

where $\bar{\Sigma}$ is a set of subterms, i.e. grammar elements that transmit the automat to the next lower level.

The rules containing subterms are the following:

$$\bar{a}_t \xrightarrow[r_0^{n+1}]{W\gamma(\gamma_1,\dots,\gamma_n)} r_m^n,$$

where r_0^{n+1} is a receiver-rules- the initial complex of next level grammar, $r_m^n$is a receiver-rules to which the transition is done when r_k^{n+1}.

As an example, the tabular form of grammar of activity of UML- diagrams is given in Table 1.

TABLE I. THE TABULAR FORM OF GRAMMAR OF UML DIAGRAMS ACTIVITY

№	Complex	Quaziterm	The complex-the successor	RV- relation
1.	r0	label	r3	∅
2.	r1	label	r3	∅
3.	r2	labelP	r3	$W_2(b^{1m})$
4.		labelW	r3	$W_2(b^{2m})$
5.		labelR	r3	$W_2(b^{3m})$
6.		labelL	r3	$W_2(b^{4m})/W_3(m^{t(2)} == k^{t(3)})$
7.	r3	A	r1	∅
8.		P	r1	$W_1(t^{1m})$
9.		W	r2	$W_1(1^{t(1)}, t^{2m})/W_2(e^{t(1)})$
10.		W	r2	$W_1(2^{t(1)})/W_2(1^{t(1)})$
11.		R	r1	$W_1(t^{3m^{(k-1)}})/W_3(k > 1)$
12.		L	r2	$W_1(1^{t(2)}, k^{t(3)}, t^{4m})/W_2(e^{t(2)})$
13.		L	r2	$W_1(\mathrm{inc}(m^{t(2)}))/W_3(m^{t(2)} < k^{t(3)})$
14.		AK	rk	∅

IV. SEMANTIC ANALYSIS AND CONTROL

While collective designing it is important to control the ontological consistency of the complex of designed diagrams. These errors are semantic. Therefore to analyze the semantic correctness is proposed to be used the multi-level grammar. The upper level of multi-level grammar is a grammar of use case diagrams of as well as the development of the complex automated system, according to the methodology RUP, starts with this chart.

In analyzing the semantic information is stored on the domain field as a graph of relations between semantic concepts (textual information), loaded to the blocks and relationship diagrams. Each new diagram is analyzed for conditions consistent domain concepts expansion.

In the construction of the first diagrams only semantic consistency within the diagram is checked: the use of concepts in the semantic pair. When you add new diagrams the diagram's consistency is separately ensured, and it is controlled for consistency of a complex model designed by the automated system.

To control the complex model, you need to construct a graph of semantic links between the elements of the automated

system. To solve this problem an adapted method of lexical and syntactic patterns has been selected.

The proposed method allows users to diagnose the following semantic errors: Large synonyms, Object's Antonyms, conversion of links. One class of the errors called Incompatible objects is not controlled by the proposed method.

The developed methods of analysis and control can diagnose errors of separate contexts as well as semantic errors, which are not defined in most modern editors.

The study has detected the following types of errors that occur in the UML diagrams (Table 2).

TABLE II. THE ERROR TYPES OF UML DIAGRAMS

№	Type of error	UD	AD	SD	CD	DD
1.	Lack of link	+	+	+	+	+
2.	Error of transfer of control					
3.	Error of multiplicity of inputs		+			+
4.	Error of multiplicity of outputs		+			
5.	Invalid link	+	+	+	+	+
6.	Error of link	+	+			+
7.	Error of access level				+	
8.	Error of message transfer		+	+		
9.	Error of delegation of management				+	
10.	Quantitative error of diagram elements		+			+
11.	Excluding links of the wrong type				+	
12.	Call to the lifeline			+		
13.	Unbounded links	+	+	+	+	+
14.	Violation of multiplicity of dependencies	+	+			
15.	Mutually exclusive relations	+				
16.	Multiple links	+				
17.	Infinite cycle	+				
18.	Circular links	+				
19.	Synchronous call until get answer			+		
20.	Error of distant context		+			

The table uses the following abbreviations: UD the use of diagram; AD- activity diagram; SD – sequence diagram, CD-class diagram, DD – deployment diagrams.

The research of modern systems of the UML diagrammatic notations development has shown that they allow one to find out the first 16 types of the above mentioned errors. The author's toolkit of RVM-grammars allows one to find out four additional types of errors: Multiple links, Circular links, Synchronous call until get answer, Error of distant context.

The error of distant context is an error that occurs in the paired elements, e.g., such as conditional branching and merging of conditional branches. This error is characterized by the possible presence of an unlimited number of units and links between them. The example of such error can be a conditional branch in the activity diagram, which is supposed the execution of only one of the possible branches and paired with the merger of parallel branches, which transfers control if only the all input connections are completed. This combination of elements never allows completing the diagram.

To analyze and control the errors the software tools have been developed: syntax-oriented analyzer of UML diagrams for MS Visio, network system of diagrammatic models analysis and control offering a full set of functionality to analyze and control syntactic and semantic errors.

The tools of the last system allow performing the following main functions for users:

- development of business-process model diagrams by using the different graphical languages;
- addition of new notations to the graphical languages;
- analysis of developed diagrams by using RVM-grammars on pre-loaded language descriptions and analysis rules into the system;
- addition of new analysis algorithms by plug-ins;
- addition of syntax and semantic rules of graphical languages for using in RVM-analyzer;
- creation of the interrelation between diagrams built by designers;
- simultaneous access of several designers to the database of designed diagrams.

V. CONCLUSION

It is proposed the method of analysis and control of diagrammic UML models errors within the complex diagrams developed in the collective design. The method is based on the finite state graphical RVM-grammars. RVM-grammars allow one to find out four additional types of errors that is 20% of the total number of errors. Tools for analysis and control of diagrammic models have been developed.

ACKNOWLEDGMENT

The reported study was partially supported by RFBR, research project №15-07-08268a.

REFERENCES

[1] Afanasyev A. N. The methodology and tools for analysis and control of workflows in computer-aided design of complex computer-based systems // Proceedings of the Congress on Intelligent Systems and Information Technology «IS & IT'12». Ed. in 4 volumes. Vol.1. - Moscow: FIZMATLIT, 2012. – pp. 391-399.

[2] Fu K. Structural methods of pattern recognition. – Moscow: Mir, 1977. – P.319.

[3] Costagliola G., Lucia A.D., Orece S., Tortora G. A parsing methodology for the implementation of visual systems.

http://www.dmi.unisa.it/people/costagliola/www/home/papers/method.ps.gz.

[4] Wittenburg K., Weitzman L. Relational grammars: Theory and practice in a visual language interface for process modeling. — 1996. http://citeseer.ist.psu.edu/wittenburg96relational.html.

[5] Zhang D. Q., Zhang K. Reserved graph grammar: A specification tool for diagrammatic VPLs //Visual Languages, 1997. Proceedings. 1997 IEEE Symposium on. – IEEE, 1997. – pp. 284-291.

[6] Zhang K. B., Zhang K., Orgun M. A. Using Graph Grammar to Implement Global Layout for A Visual Programming Language Generation System. – 2002.